EXPERIMENT 1: STUDENT–COLLEGE DATABASE

Self-contained SQL practical • MySQL 8.0+ • Create database, tables and sample data included

```
-- EXPERIMENT 1: Create the complete Student–College database
CREATE DATABASE IF NOT EXISTS college;
USE college;

CREATE TABLE department (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(40) NOT NULL UNIQUE,
    hod_name VARCHAR(50)
);

CREATE TABLE student (
    roll_no INT PRIMARY KEY,
    student_name VARCHAR(50) NOT NULL,
    age INT NOT NULL,
    email VARCHAR(60) UNIQUE,
    dept_id INT NOT NULL,
    FOREIGN KEY (dept_id) REFERENCES department(dept_id)
);

CREATE TABLE course (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(50) NOT NULL,
    credits INT NOT NULL
);

CREATE TABLE enrollment (
    roll_no INT,
    course_id INT,
    marks INT,
    PRIMARY KEY (roll_no, course_id),
    FOREIGN KEY (roll_no) REFERENCES student(roll_no),
    FOREIGN KEY (course_id) REFERENCES course(course_id)
);

INSERT INTO department VALUES
(1, 'Computer', 'Patil'),
(2, 'Information Technology', 'Shah'),
(3, 'Mechanical', 'Joshi');

INSERT INTO student VALUES
(101, 'Amit', 20, 'amit@example.com', 1),
(102, 'Neha', 21, 'neha@example.com', 2),
(103, 'Rahul', 19, 'rahul@example.com', 1),
(104, 'Kiran', 20, 'kiran@example.com', 3);

INSERT INTO course VALUES
(11, 'DBMS', 4),
(12, 'Python', 3),
(13, 'Web Technology', 3);

INSERT INTO enrollment VALUES
(101, 11, 85), (101, 12, 78), (102, 11, 90),
(103, 13, 74), (104, 12, 81);

SELECT * FROM department;
SELECT * FROM student;
SELECT * FROM course;
SELECT * FROM enrollment;
```

EXPERIMENT 2: ER/EER TO RELATIONAL SCHEMA

Self-contained SQL practical • MySQL 8.0+ • Create database, tables and sample data included

```
-- EXPERIMENT 2: Map the ER/EER model to relations
CREATE DATABASE IF NOT EXISTS college;
USE college;

CREATE TABLE department (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(40) NOT NULL UNIQUE
);

CREATE TABLE student (
    roll_no INT PRIMARY KEY,
    student_name VARCHAR(50) NOT NULL,
    age INT,
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES department(dept_id)
);

CREATE TABLE course (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(50) NOT NULL,
    credits INT
);

-- M:N relationship between STUDENT and COURSE
CREATE TABLE enrollment (
    roll_no INT,
    course_id INT,
    marks INT,
    PRIMARY KEY (roll_no, course_id),
    FOREIGN KEY (roll_no) REFERENCES student(roll_no),
    FOREIGN KEY (course_id) REFERENCES course(course_id)
);

INSERT INTO department VALUES (1, 'Computer'), (2, 'IT');

INSERT INTO student VALUES
(101, 'Amit', 20, 1), (102, 'Neha', 21, 2), (103, 'Rahul', 19, 1);

INSERT INTO course VALUES (11, 'DBMS', 4), (12, 'Python', 3);

INSERT INTO enrollment VALUES
(101, 11, 85), (101, 12, 78), (102, 11, 90);

-- Schema:
-- DEPARTMENT(dept_id PK, dept_name)
-- STUDENT(roll_no PK, student_name, age, dept_id FK)
-- COURSE(course_id PK, course_name, credits)
-- ENROLLMENT(roll_no PK/FK, course_id PK/FK, marks)

SELECT * FROM department;
SELECT * FROM student;
SELECT * FROM course;
SELECT * FROM enrollment;
```

EXPERIMENT 3: DDL AND INTEGRITY CONSTRAINTS

Self-contained SQL practical • MySQL 8.0+ • Create database, tables and sample data included

```
-- EXPERIMENT 3: DDL and integrity constraints
CREATE DATABASE IF NOT EXISTS college;
USE college;

CREATE TABLE department (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(40) NOT NULL UNIQUE
);

CREATE TABLE student (
    roll_no INT PRIMARY KEY,
    student_name VARCHAR(50) NOT NULL,
    email VARCHAR(60) UNIQUE,
    age INT CHECK (age >= 16),
    dept_id INT,
    status VARCHAR(15) DEFAULT 'Active',
    FOREIGN KEY (dept_id) REFERENCES department(dept_id)
);

INSERT INTO department VALUES (1, 'Computer'), (2, 'IT');

INSERT INTO student (roll_no, student_name, email, age, dept_id)
VALUES (101, 'Amit', 'amit@example.com', 20, 1),
       (102, 'Neha', 'neha@example.com', 21, 2);

ALTER TABLE student ADD phone VARCHAR(15);
ALTER TABLE student MODIFY student_name VARCHAR(60) NOT NULL;
ALTER TABLE student ADD CONSTRAINT chk_student_age CHECK (age >= 16);
ALTER TABLE student DROP CHECK chk_student_age;
ALTER TABLE student DROP COLUMN phone;

CREATE TABLE student_test (
    test_id INT PRIMARY KEY,
    test_name VARCHAR(30) NOT NULL
);

INSERT INTO student_test VALUES (1, 'Sample');
RENAME TABLE student_test TO learner_test;
TRUNCATE TABLE learner_test;
DROP TABLE learner_test;

SHOW TABLES;
SELECT * FROM student;
```

EXPERIMENT 4: DML — INSERT, UPDATE, DELETE

Self-contained SQL practical • MySQL 8.0+ • Create database, tables and sample data included

```
-- EXPERIMENT 4: DML commands
CREATE DATABASE IF NOT EXISTS college;
USE college;

CREATE TABLE department (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(40) NOT NULL
);

CREATE TABLE student (
    roll_no INT PRIMARY KEY,
    student_name VARCHAR(50) NOT NULL,
    age INT,
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES department(dept_id)
);

INSERT INTO department VALUES
(1, 'Computer'), (2, 'IT'), (3, 'Mechanical');

INSERT INTO student VALUES
(101, 'Amit', 20, 1), (102, 'Neha', 21, 2),
(103, 'Rahul', 19, 1), (104, 'Kiran', 20, 3);

INSERT INTO student (roll_no, student_name, age, dept_id)
VALUES (105, 'Sonal', 22, 2);
SELECT * FROM student;

UPDATE student SET age = 23 WHERE roll_no = 105;
SELECT * FROM student;

DELETE FROM student WHERE roll_no = 103;
SELECT * FROM student;
```

EXPERIMENT 5: DCL — GRANT AND REVOKE

Self-contained SQL practical • MySQL 8.0+ • Create database, tables and sample data included

```
-- EXPERIMENT 5: DCL commands
CREATE DATABASE IF NOT EXISTS college;
USE college;

CREATE TABLE department (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(40) NOT NULL
);

CREATE TABLE student (
    roll_no INT PRIMARY KEY,
    student_name VARCHAR(50) NOT NULL,
    age INT,
    dept_id INT
);

INSERT INTO department VALUES (1, 'Computer'), (2, 'IT');
INSERT INTO student VALUES (101, 'Amit', 20, 1), (102, 'Neha', 21, 2);

-- User creation may require administrator permission
CREATE USER IF NOT EXISTS 'college_user'@'localhost'
IDENTIFIED BY 'College123';

GRANT SELECT, INSERT, UPDATE
ON college.student TO 'college_user'@'localhost';

SHOW GRANTS FOR 'college_user'@'localhost';

REVOKE INSERT, UPDATE
ON college.student FROM 'college_user'@'localhost';

SHOW GRANTS FOR 'college_user'@'localhost';

-- Optional cleanup after practical:
-- DROP USER 'college_user'@'localhost';
```

EXPERIMENT 6: SQL QUERIES AND AGGREGATE FUNCTIONS

Self-contained SQL practical • MySQL 8.0+ • Create database, tables and sample data included

```
-- EXPERIMENT 6: SELECT queries and aggregate functions
CREATE DATABASE IF NOT EXISTS college;
USE college;

CREATE TABLE department (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(40) NOT NULL
);

CREATE TABLE student (
    roll_no INT PRIMARY KEY,
    student_name VARCHAR(50) NOT NULL,
    age INT,
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES department(dept_id)
);

INSERT INTO department VALUES
(1, 'Computer'), (2, 'IT'), (3, 'Mechanical');

INSERT INTO student VALUES
(101, 'Amit', 20, 1), (102, 'Neha', 21, 2),
(103, 'Rahul', 19, 1), (104, 'Kiran', 20, 3),
(105, 'Sonal', 22, 2);

SELECT * FROM student;
SELECT student_name, age FROM student;
SELECT * FROM student WHERE age >= 20;
SELECT * FROM student WHERE age >= 20 AND dept_id = 1;
SELECT * FROM student WHERE dept_id = 1 OR dept_id = 2;
SELECT * FROM student WHERE age BETWEEN 19 AND 21;
SELECT * FROM student WHERE dept_id IN (1, 2);
SELECT * FROM student WHERE student_name LIKE 'A%';

SELECT DISTINCT dept_id FROM student;
SELECT * FROM student ORDER BY student_name ASC;
SELECT * FROM student ORDER BY age DESC;

SELECT COUNT(*) AS total_students FROM student;
SELECT SUM(age) AS total_age FROM student;
SELECT AVG(age) AS average_age FROM student;
SELECT MIN(age) AS minimum_age FROM student;
SELECT MAX(age) AS maximum_age FROM student;

SELECT dept_id, COUNT(*) AS student_count
FROM student GROUP BY dept_id;

SELECT dept_id, AVG(age) AS average_age
FROM student GROUP BY dept_id
HAVING AVG(age) >= 20;
```

EXPERIMENT 7: JOIN OPERATIONS

Self-contained SQL practical • MySQL 8.0+ • Create database, tables and sample data included

```
-- EXPERIMENT 7: Join operations
CREATE DATABASE IF NOT EXISTS college;
USE college;

CREATE TABLE department (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(40) NOT NULL
);

CREATE TABLE student (
    roll_no INT PRIMARY KEY,
    student_name VARCHAR(50) NOT NULL,
    age INT,
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES department(dept_id)
);

INSERT INTO department VALUES
(1, 'Computer'), (2, 'IT'), (3, 'Mechanical'), (4, 'Civil');

INSERT INTO student VALUES
(101, 'Amit', 20, 1), (102, 'Neha', 21, 2),
(103, 'Rahul', 19, 1), (104, 'Kiran', 20, 3);

-- Equijoin / INNER JOIN
SELECT student.student_name, department.dept_name
FROM student INNER JOIN department
ON student.dept_id = department.dept_id;

-- Non-equijoin
SELECT student.student_name, student.age, department.dept_name
FROM student JOIN department
ON student.age >= 20 AND department.dept_id <= 2;

-- Self-join: students in the same department
SELECT a.student_name AS student1, b.student_name AS student2, a.dept_id
FROM student a JOIN student b
ON a.dept_id = b.dept_id AND a.roll_no < b.roll_no;

-- LEFT OUTER JOIN
SELECT student.student_name, department.dept_name
FROM student LEFT JOIN department
ON student.dept_id = department.dept_id;

-- RIGHT OUTER JOIN
SELECT student.student_name, department.dept_name
FROM student RIGHT JOIN department
ON student.dept_id = department.dept_id;

-- CROSS JOIN
SELECT student.student_name, department.dept_name
FROM student CROSS JOIN department;
```

EXPERIMENT 8: VIEWS AND TRIGGERS

Self-contained SQL practical • MySQL 8.0+ • Create database, tables and sample data included

```
-- EXPERIMENT 8: Create a view and audit triggers
CREATE DATABASE IF NOT EXISTS college;
USE college;

CREATE TABLE department (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(40) NOT NULL
);

CREATE TABLE student (
    roll_no INT PRIMARY KEY,
    student_name VARCHAR(50) NOT NULL,
    age INT,
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES department(dept_id)
);

INSERT INTO department VALUES (1, 'Computer'), (2, 'IT');
INSERT INTO student VALUES
(101, 'Amit', 20, 1), (102, 'Neha', 21, 2), (103, 'Rahul', 19, 1);

CREATE VIEW student_view AS
SELECT roll_no, student_name, dept_id FROM student;

SELECT * FROM student_view;

CREATE TABLE student_log (
    log_id INT AUTO_INCREMENT PRIMARY KEY,
    action_name VARCHAR(20) NOT NULL,
    roll_no INT,
    action_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

DELIMITER //

CREATE TRIGGER student_after_insert
AFTER INSERT ON student FOR EACH ROW
BEGIN
    INSERT INTO student_log (action_name, roll_no)
    VALUES ('INSERT', NEW.roll_no);
END//

CREATE TRIGGER student_after_update
AFTER UPDATE ON student FOR EACH ROW
BEGIN
    INSERT INTO student_log (action_name, roll_no)
    VALUES ('UPDATE', NEW.roll_no);
END//

CREATE TRIGGER student_after_delete
AFTER DELETE ON student FOR EACH ROW
BEGIN
    INSERT INTO student_log (action_name, roll_no)
    VALUES ('DELETE', OLD.roll_no);
END//

DELIMITER ;

INSERT INTO student VALUES (104, 'Sonal', 22, 1);
UPDATE student SET age = 23 WHERE roll_no = 104;
DELETE FROM student WHERE roll_no = 104;

SELECT * FROM student_log;
SHOW TRIGGERS;

-- Optional cleanup commands:
-- DROP VIEW student_view;
-- DROP TRIGGER student_after_insert;
-- DROP TRIGGER student_after_update;
-- DROP TRIGGER student_after_delete;
```

EXPERIMENT 9: TRANSACTIONS AND CONCURRENCY CONTROL

Self-contained SQL practical • MySQL 8.0+ • Create database, tables and sample data included

```
-- EXPERIMENT 9: COMMIT, ROLLBACK and row locking
CREATE DATABASE IF NOT EXISTS college;
USE college;

CREATE TABLE department (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(40) NOT NULL
);

CREATE TABLE student (
    roll_no INT PRIMARY KEY,
    student_name VARCHAR(50) NOT NULL,
    age INT,
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES department(dept_id)
) ENGINE = InnoDB;

INSERT INTO department VALUES (1, 'Computer'), (2, 'IT');
INSERT INTO student VALUES
(101, 'Amit', 20, 1), (102, 'Neha', 21, 2), (103, 'Rahul', 19, 1);

START TRANSACTION;
INSERT INTO student VALUES (104, 'Kiran', 20, 1);
UPDATE student SET age = 21 WHERE roll_no = 104;
SELECT * FROM student WHERE roll_no = 104;
COMMIT;
SELECT * FROM student WHERE roll_no = 104;

START TRANSACTION;
UPDATE student SET age = 25 WHERE roll_no = 104;
SELECT * FROM student WHERE roll_no = 104;
ROLLBACK;
SELECT * FROM student WHERE roll_no = 104;

-- Concurrency demo: run the following in Session 1
START TRANSACTION;
SELECT * FROM student WHERE roll_no = 104 FOR UPDATE;

-- Keep Session 1 transaction open.
-- In Session 2, run:
-- UPDATE student SET age = 22 WHERE roll_no = 104;
-- Session 2 waits because Session 1 has locked the row.

-- Back in Session 1, release the lock:
COMMIT;

-- Session 2 update can now finish. Check:
SELECT * FROM student WHERE roll_no = 104;
```